

2-TP2

Dupla: Arthur Bicalho e Fernando Leite Pagani

(i) Código de submissão

```
#include <stdio.h>

void solve() {
    int n;

    // Lê a quantidade de cartas
    scanf("%d", &n);

    // Declarando os arrays com espaço de sobra para as 1000 cartas + o '\0'
    char red[1005];
    char blue[1005];

    // Lê as duas sequências de dígitos contínuas como texto
    scanf("%s", red);
    scanf("%s", blue);
    int red_pts = 0;
    int blue_pts = 0;

    // A associação acontece naturalmente aqui no índice 'i'.
    // red[i] e blue[i] representam a frente e o verso da mesma carta!
    for (int i = 0; i < n; i++) {
        // Tabela ASCII garante que '7' > '5' será avaliado corretamente
        if (red[i] > blue[i]) {
            red_pts++;
        }
        else if (blue[i] > red[i]) {
            blue_pts++;
        }
    }

    // Decisão final
    if (red_pts > blue_pts) {
        printf("RED\n");
    }
    else if (blue_pts > red_pts) {
        printf("BLUE\n");
    }
    else {
        printf("EQUAL\n");
    }
}
```

```

int main() {
    int t;
    scanf("%d", &t); // Casos de teste
    while (t--) {
        solve();
    }
    return 0;
}

```

(ii) Screenshot de aceite

General

#	Author	Problem	Lang	Verdict	Time	Memory	Sent	Judged		
377699850	Practice: nietzsche_bica	1459A - 8	GNU C11	Accepted	15 ms	84 KB	2026-06-07 22:24:40	2026-06-07 22:24:40	★	Compare

→ Source
Copy

```

#include <stdio.h>

void solve() {
    int n;
    // Lê a quantidade de cartas
    scanf("%d", &n);

    // Declarando os arrays com espaço de sobra para as 1000 cartas + o '\0'
    char red[1005];
    char blue[1005];

    // Lê as duas sequências de dígitos contínuas como texto
    scanf("%s", red);
    scanf("%s", blue);

    int red_pts = 0;
    int blue_pts = 0;

    // A associação acontece naturalmente aqui no índice 'i'.
    // red[i] e blue[i] representam a frente e o verso da mesma carta!
    for (int i = 0; i < n; i++) {
        // Tabela ASCII garante que '7' > '5' será avaliado corretamente
        if (red[i] > blue[i]) {
            red_pts++;
        } else if (blue[i] > red[i]) {
            blue_pts++;
        }
    }

    // Decisão final
    if (red_pts > blue_pts) {
        printf("RED\n");
    } else if (blue_pts > red_pts) {
        printf("BLUE\n");
    } else {
        printf("EQUAL\n");
    }
}

int main() {
    int t;
    scanf("%d", &t); // Casos de teste
    while (t--) {
        solve();
    }
    return 0;
}

```

(iii) Manuscrito

Análise e justificativa de implementação do código. $(n) \Theta = (n) T : \text{análise}$ P.2

Uma carta com $r_i = b_i$ nunca pode influenciar o resultado. Porque, ao comparar R e B dígitos a dígitos da esquerda para a direita, essas cartas contribuirão com o mesmo dígito para ambos os números na mesma posição.

Ao comparar dois números de n dígitos, o resultado é determinado pelo primeiro índice k da esquerda onde os dígitos diferem. Portanto, apenas as cartas onde $r_i \neq b_i$ são relevantes. - $D = \{i : r_i \neq b_i\}$. $8 = 8$

$$\text{subarray } 02 + 08 = (n) T = (n) T$$

Análise de complexidade

5.1

```
for (int i = 0; i < n; i++) {
```

```
    if (red[i] > blue[i]) {
```

```
        red_Pts++;
```

```
    } else if (blue[i] > red[i]) {
```

```
        blue_Pts++;
```

```
    }
```

```
    // + 1 incremento do índice i++
```

```
    // + 1 verificação i < n
```

```
}
```

// 2 acessos a array + 1 comparação

// 1 incremento (no pior caso)

// 2 acessos a array + 1 comparação

// 1 incremento (no pior caso) - (local) +

$001 = n$ e $001 = T$: duplo acesso ao ra. E.2

2 em 8 = $2^6 \cdot 01 \cdot 8 = 2^6 \cdot 01 \cdot 8$, $001 = (n) T$, $T = 6 + 1 + 1$

Portanto, cada iteração executa no máximo $K=8$ operações. Com C_0 sendo uma constante que engloba a leitura de entrada e a lógica de saída, o custo total por caso de teste é:

$$T(n) = K \cdot n + C_0$$

5.2 Para todo $n \geq 1$, temos:

$$K \cdot n + C_0 \leq K \cdot n + C_0 \cdot n = (K + C_0) \cdot n$$

$$C_1 = K + C_0 \text{ e } n_0 = 1 \therefore T(n) \leq C_1 \cdot n \text{ para todo } n \geq n_0.$$

$$\therefore T(n) \in O(n).$$

5.3 $T(n) \in \Omega(n)$

Analogamente, para todo $n \geq 1$

$$T(n) = K \cdot n + C_0 \geq K \cdot n$$

Tomando $C_2 = K$ e $n_0 = 1$, temos $T(n) \geq C_2 \cdot n$, o que satisfaz a definição de $\Omega(n)$

5.4 Conclusão: $T(n) \in \Theta(n)$

Como $T(n) \in O(n)$ e $T(n) \in \Omega(n)$, concluímos que $T(n) \in \Theta(n)$: onde a solução é linearmente proporcional aos números de centros.

6. Cálculo do Tempo de Execução

6.1 Tempo em função de n

$$K=8$$

$$t(n) = T(n) = \frac{8n + C_0}{10^8} \text{ segundos}$$

Desconsiderando a constante C_0 , temos a fórmula simplificada:

$$t(n) \approx \frac{8n}{10^8} = \frac{n}{12,5 \cdot 10^6} \text{ segundos}$$

6.2 Avaliação dos limites do problema

Em um único caso de teste com $n=1000$ (valor máximo):

$$t(1000) = \frac{8 \cdot 1000}{10^8} = \frac{8 \cdot 10^3}{10^8} = 8 \cdot 10^{-5} \text{ s} = 80 \mu\text{s}$$

6.3 Por caso global: $T=100$ e $n=1000$

O programa pode receber até $T=100$ casos de teste. O custo total é:

$$t_{\text{total}} = T \cdot t(n) = 100 \cdot 8 \cdot 10^{-5} = 8 \cdot 10^{-3} \text{ s} = 8 \text{ ms}$$

7 Resumo

Complexidade por caso de teste: $\Theta(n)$

Complexidade global (pior caso): $\Theta(T \cdot n)$

Tempo máximo por caso $\approx 80 \mu\text{s}$

Tempo máximo total $\approx 8 \text{ ms}$